

Университет ИТМО

Факультет программной инженерии и компьютерной техники

Лабораторная работа №4
по «Алгоритмам и структурам данных»
Базовые задачи

Выполнила:

Студентка группы Р3215

Павличенко Софья Алексеевна

Преподаватели:

Косяков М.С.

Тараканов Д.С.

Санкт-Петербург

2025

Задача М «Цивилизация»

Код:

```
1 #include <deque>
2 #include <iostream>
3 #include <vector>
4
5 using namespace std;
6
7 int main() {
8     int n, m, x1, y1, x2, y2;
9     cin >> n >> m >> x1 >> y1 >> x2 >> y2;
10    x1--;
11    y1--;
12    x2--;
13    y2--;
14    vector<vector<char>> mat(n, vector<char>(m));
15    vector<vector<pair<int, int>>> g(n * m);
16    vector<pair<int, char>> d(n * m);
17
18    for (int i = 0; i < n; ++i) {
19        for (int j = 0; j < m; ++j) {
20            cin >> mat[i][j];
21            d[i * m + j] = {1e9, '.'};
22        }
23    }
24    for (int i = 0; i < n; ++i) {
25        for (int j = 0; j < m; ++j) {
26            if (mat[i][j] == '#')
27                continue;
28            if (j + 1 < m && mat[i][j + 1] != '#')
29                g[i * m + j].push_back({i * m + j + 1, mat[i][j + 1] == '.' ? 1 : 2});
30            if (j - 1 >= 0 && mat[i][j - 1] != '#')
31                g[i * m + j].push_back({i * m + j - 1, mat[i][j - 1] == '.' ? 1 : 2});
32            if (i + 1 < n && mat[i + 1][j] != '#')
33                g[i * m + j].push_back({(i + 1) * m + j, mat[i + 1][j] == '.' ? 1 : 2});
34            if (i - 1 >= 0 && mat[i - 1][j] != '#')
35                g[i * m + j].push_back({(i - 1) * m + j, mat[i - 1][j] == '.' ? 1 : 2});
36        }
37    }
38
39    deque<int> q;
40    q.push_front(x1 * m + y1);
41
42    d[x1 * m + y1] = {0, '.'};
43
44    while (!q.empty()) {
45        int v = q.front();
46        q.pop_front();
47
48        for (auto [u, c] : g[v]) {
49            if (d[v].first + c < d[u].first) {
50                d[u].first = d[v].first + c;
51                if (u - v == 1)
52                    d[u].second = 'E';
53                else if (v - u == 1)
54                    d[u].second = 'W';
55                else if (u - v == m)
56                    d[u].second = 'S';
57                else
58                    d[u].second = 'N';
59
60                if (c == 1)
61                    q.push_front(u);
62                else
63                    q.push_back(u);
64            }
65        }
66    }
67
68    if (d[x2 * m + y2].first == 1e9) {
69        cout << -1;
70        return 0;
71    }
72    cout << d[x2 * m + y2].first << endl;
73    string res;
74    int u = x2 * m + y2;
75    while (u != x1 * m + y1) {
76        res += d[u].second;
77        if (d[u].second == 'E')
78            u--;
79        else if (d[u].second == 'W')
80            u++;
81        else if (d[u].second == 'S')
82            u -= m;
83        else
84            u += m;
85    }
86    for (int i = res.size() - 1; i >= 0; --i)
87        cout << res[i];
88    return 0;
89 }
90
```

Пояснение к примененному алгоритму:

Алгоритм использует модифицированный 0-1 BFS для поиска кратчайшего пути с разными весами перехода (1 — поле, 2 — лес). Клетки представляют вершины графа, соседние проходимые клетки — рёбра с весами. Более дешёвые переходы добавляются в начало очереди, дорогие — в конец, что позволяет эффективно обновлять минимальное расстояние. Восстановление пути производится по направлению, сохранённому при обновлении.

Время — $O(NM)$, так как каждая клетка обрабатывается не более двух раз.

Память — $O(NM)$, на хранение карты, графа и расстояний.

Задача О «Долой списывание!»

Код:

```
1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 vector<int> colors;
7 vector<vector<int>> vs;
8
9 void dfs(int v, int color) {
10     colors[v] = color;
11     for (int u : vs[v]) {
12         if (colors[u] == 0)
13             dfs(u, 3 - color);
14         else if (colors[u] == color) {
15             cout << "NO";
16             exit(0);
17         }
18     }
19 }
20
21 int main() {
22     int n, m, u1, u2;
23     cin >> n >> m;
24     vs.resize(n + 1);
25     colors.resize(n + 1, 0);
26     for (int i = 0; i < m; ++i) {
27         cin >> u1 >> u2;
28         vs[u1].push_back(u2);
29         vs[u2].push_back(u1);
30     }
31     for (int u = 1; u <= n; ++u) {
32         if (colors[u] == 0)
33             dfs(u, 1);
34     }
35     cout << "YES";
36     return 0;
37 }
38
```

Пояснение к примененному алгоритму:

Алгоритм проверяет, двудолен ли граф: можно ли раскрасить вершины в два цвета так, чтобы смежные имели разные цвета. Для этого используется DFS с покраской. Если во время обхода встречаются две вершины одного цвета, выводится «NO». Иначе, после обхода всех компонент — «YES».

Время — $O(N + M)$, так как каждая вершина и ребро посещается один раз.

Память — $O(N + M)$, на хранение графа и цветов.

Задача Р «Авиаперелёты»

Код:

```
1
2 #include <iostream>
3 #include <vector>
4
5 using namespace std;
6
7 int n;
8 vector<vector<int>> graph;
9 vector<vector<bool>> new_graph;
10 vector<bool> used;
11
12 void dfs(int v, bool reversed) {
13     used[v] = true;
14     for (int u = 0; u < n; ++u) {
15         if (!used[u] && ((reversed && new_graph[u][v]) || (!reversed && new_graph[v][u])))
16             dfs(u, reversed);
17     }
18 }
19
20 int main() {
21     int l = -1, r = 0, m;
22     cin >> n;
23
24     graph.assign(n, vector<int>(n));
25     new_graph.assign(n, vector<bool>(n));
26     used.assign(n, false);
27
28     for (int i = 0; i < n; ++i) {
29         for (int j = 0; j < n; ++j) {
30             cin >> graph[i][j];
31             r = max(r, graph[i][j]);
32         }
33     }
34
35     while (r - l > 1) {
36         m = (l + r) / 2;
37         for (int i = 0; i < n; ++i) {
38             for (int j = 0; j < n; ++j) {
39                 new_graph[i][j] = graph[i][j] <= m;
40             }
41         }
42         fill(used.begin(), used.end(), false);
43         dfs(0, false);
44         bool flag = true;
45         for (int u = 0; u < n; ++u) {
46             if (!used[u])
47                 flag = false;
48         }
49         if (flag) {
50             fill(used.begin(), used.end(), false);
51             dfs(0, true);
52             for (int u = 0; u < n; ++u) {
53                 if (!used[u])
54                     flag = false;
55             }
56         }
57         if (flag)
58             r = m;
59         else
60             l = m;
61     }
62     cout << r;
63     return 0;
64 }
65
```

Пояснение к примененному алгоритму:

Алгоритм использует бинарный поиск по ответу: минимальному баку, при котором граф становится сильно связным (из любого города можно добраться в любой). На каждой итерации строится вспомогательный граф с рёбрами $\leq$ текущего значения, и проверяется сильная связность с помощью двух DFS — прямого и обратного.

Время — $O(N^2 \log C)$, где C — максимальное значение в матрице.

Память — $O(N^2)$, для хранения графов и посещённых вершин.